

國立中央大學

114學年度 第一學期

軟體工程 期末書面報告

C 軟體設計文件(架構設計)

寵物領養平台

組別：第十七組

組員：

114423001 陳柏甫

114423007 潘維晟

114423042 劉和媛

114423043 李嶧

目錄

1. 系統架構概述	3
1.1 表示層 (Presentation Layer)	3
1.2 應用層 (Application Layer)	4
1.3 資料層 (Data Layer)	4
1.4 資料流說明	4
2. 模組分解	6
2.1 前端	6
2.2 後端	8
3. 資料庫設計	12
3.1 資料表	12
3.3 資料表間的關聯關係	24
3.4 ERD	25
4. 系統架構圖	26

1. 系統架構概述

本系統採用三層式架構（Presentation / Application / Data），下列為每一層的技术選擇與主要職責：

1.1 表示層（Presentation Layer）

a. 技術選擇：

- i. 框架：Vue 3 + Vite + TypeScript (SPA)
- ii. 狀態管理：Pinia；Server-state: @tanstack/vue-query
- iii. 表單驗證：vee-validate + zod
- iv. UI樣式：Tailwind CSS

b. 主要職責：

- i. 呈現使用者介面與互動。
- ii. 處理表單驗證、草稿快取與本地輸入暫存。
- iii. 與後端 REST API 交換資料（HTTP/JSON）：發起請求、顯示回應、錯誤處理與 retry 策略。
- iv. 實作 presign 上傳流程：
 1. 前端向 API 取得 presigned URL。
 2. 前端直接上傳檔案至 Object Storage。
 3. 上傳完成後回報 metadata。
- v. 顯示長時間任務狀態與通知（/jobs/{jobId} 輪詢或透過 websocket / pubsub）。
- vi. 客戶端路由守衛與基本存取導向（基於角色或所有權的 UI 控制）。

c. 主要功能頁面：

- i. Landing — 平台首頁、導向登入或主要功能入口。
- ii. Listings — 動物清單搜尋與篩選。
- iii. Listing Detail — 動物詳細資訊、圖片與申請入口。
- iv. My Applications — 顯示使用者申請紀錄與狀態。（一般會員）
- v. My Rehomes — 管理使用者的刊登資料，並進行審核。（一般會員）
- vi. Shelter Dashboard — 收容所會員專屬管理介面（刊登、申請審核）
- vii. Admin Dashboard — 管理員後台，用於審核與系統監控。

d. 與其他層互動：

- i. 透過 HTTPS REST API 與應用層（Application Layer）交換資料；上傳流程為 presign → 直傳 MinIO (Object Storage物件儲存系統) → 回傳 metadata 給 API。
- ii. 接收 JSON 資料並更新前端狀態與 UI 呈現。

1.2 應用層 (Application Layer)

- a. 技術選擇：Flask (Blueprints 組織)，使用 flask-smorest 以便與 OpenAPI contract 整合。
- b. 主要職責：
 - i. 提供清晰且可版本化的 RESTful API (遵循 OpenAPI contract)。
 - ii. 驗證 (authentication) 與授權 (authorization / RBAC / ownership checks)。
 - iii. 實作業務邏輯並管理資料一致性與交易。
 - iv. 管理檔案中繼資料 (File Metadata)。
 - v. 管理附件 metadata、生成 presigned URLs 並驗證上傳擁有權。
 - vi. 對外排程長時間任務 (enqueue job -> 202 + jobId) 並監控 job lifecycle。
 - vii. 寫入 AuditLog、處理 idempotency 與錯誤重試策略。
- c. 背景任務處理 (Worker)：
Celery + Redis 負責處理長時間任務 (batch import/export、檔案處理、外部系統整合)；API 使用 202 + jobId 模式回應並將工作放入 Redis 佇列。

1.3 資料層 (Data Layer)

- a. 資料庫：MySQL (InnoDB)，使用 SQLAlchemy (Declarative) 與 Alembic 管理 schema 與 migrations。
- b. 物件儲存 (Object Storage)：採用 MinIO：開發可在 docker-compose 使用單節點 MinIO 快速啟動，生產則採用 MinIO 分散式模式或透過 MinIO Operator / Helm 在 Kubernetes 部署以確保可用性與擴充性。

1.4 資料流說明

- a. 整體資料流如下：
 - i. 前端 (Vue3) 透過 HTTPS 向後端 Flask API 發送請求 (JSON body / Bearer token)。
 - ii. API 層負責：
 1. 驗證請求身份與權限；
 2. 執行業務邏輯；
 3. 存取 MySQL；
 4. 處理 presigned upload 與 attachments metadata；
 5. 若為長時間任務，將工作排入 Celery/Redis 佇列，並立即回傳 HTTP 202 (Accepted) 與 jobId。
 - iii. 背景工作處理器 (Celery workers)：

從佇列中取出任務，進行非同步處理（如 CSV 匯入、圖片處理、外部 API 呼叫），並於完成後更新資料庫與 job/notification 狀態。

2. 模組分解

2.1 前端

a. 設計原則：

- Feature-first：以 domain/feature 為單位拆分模組，模組內包含 UI、composables、types、stories、tests。
- Contract-first：OpenAPI 作為 source-of-truth；前端使用 open api-typescript 產出 types，並用 msw 建立 mock handlers。
- 單向資料流：components (presentational) 與 composables/stories (data & effects) 分離。
- 可測試：每個 composable 有 unit tests；主要流程（發佈、申請、上傳）有 E2E 覆蓋。

b. 前端專案結構：

```
frontend/
├── src/
│   ├── api/                                # API 客戶端模組
│   │   ├── animals.ts                      # 動物 API
│   │   ├── applications.ts                 # 申請 API
│   │   ├── auth.ts                         # 認證 API
│   │   ├── medical.ts                     # 醫療 API
│   │   ├── notifications.ts               # 通知 API
│   │   └── users.ts                        # 使用者 API
│   ├── components/                          # Vue 組件
│   │   ├── AnimalCard.vue
│   │   ├── NotificationBell.vue
│   │   ├── FileUploader.vue
│   │   └── ...
│   ├── composables/                         # Composition API (可重用邏輯)
│   │   ├── useAuth.ts
│   │   ├── useNotifications.ts
│   │   └── ...
│   ├── pages/                              # 頁面組件
│   │   ├── Home.vue
│   │   ├── AnimalList.vue
│   │   ├── RehomeForm.vue
│   │   ├── AdminDashboard.vue
│   │   └── ...
│   ├── router/                             # Vue Router 路由配置
│   │   └── index.ts
│   ├── stores/                             # Pinia 狀態管理
│   │   ├── auth.ts                        # 認證狀態
│   │   ├── notification.ts                # 通知狀態
│   │   └── ...
│   ├── types/                              # TypeScript 類型定義
│   │   ├── animal.ts
│   │   ├── application.ts
│   │   └── ...
│   ├── App.vue                             # 根組件
│   └── main.ts                             # 應用入口
├── tests/                                  # 測試文件
│   └── e2e/                                # E2E 測試 (Playwright)
├── playwright-report/                      # 測試報告
├── index.html                             # HTML 入口
├── vite.config.ts                          # Vite 配置
├── tailwind.config.js                      # Tailwind CSS 配置
├── tsconfig.json                           # TypeScript 配置
├── package.json                            # Node.js 依賴
└── .env                                    # 環境變數
```

c. 前端模組分解：

(1) Auth

- 責任：登入/登出、token 管理、route-guards
- 檔案：‘stores/useAuth.ts’、‘composables/useAuth.ts’、‘components/auth/LoginForm.vue’
- Endpoint：POST /auth/login, POST /auth/refresh

(2) Browsing (Animals / Listings)

- 責任：列表、篩選、分頁/無限滾動、快取策略
- 檔案：‘pages/Animals.vue’、‘composables/useInfiniteAnimals.ts’、‘components/AnimalCard.vue’
- Endpoint：GET /animals?...

(3) Animal Detail & Rehome

- 責任：詳情頁、Rehome 表單（含 upload presign）、申請按鈕
- 檔案：‘pages/AnimalDetail.vue’、‘pages/RehomeForm.vue’、‘composables/useUploadPresign.ts’
- Endpoint：GET /animals/{id}, POST /rehomes

(4) Uploads / Attachments

- 責任：取得 presign URL、執行直接上傳、POST metadata、提供上傳 progress 與 retry
- 檔案：‘composables/useUploadPresign.ts’、‘components/FileUploader.vue’、‘api/uploads.ts’
- Endpoint：POST /uploads/presign, POST /attachments

(5) Applications

- 責任：提交與查詢申請、idempotency header 管理、顯示 409 錯誤 UX
- 檔案：‘pages/Applications.vue’、‘composables/useApplication.ts’
- Endpoint：POST /applications

(6) Jobs / Admin / Notifications

- 責任：job polling、admin pages、通知中心
- 檔案：‘composables/useJobStatus.ts’、‘components/JobProgress.vue’、‘stores/useNotifications.ts’
- Endpoint：GET /jobs/{jobId}, GET /notifications

2.2 後端

a. 後端模組分解

```
backend/
├── app/                                # 應用主目錄
│   ├── __init__.py                    # Flask 應用初始化
│   ├── celery.py                      # Celery 配置 (背景任務)
│   ├── exceptions.py                  # 自定義異常
│   ├── blueprints/                    # API 路由模組
│   │   ├── admin.py                  # 管理員 API
│   │   ├── animals.py                # 動物管理 API
│   │   ├── applications.py           # 申請管理 API
│   │   ├── auth.py                   # 認證 API
│   │   ├── jobs.py                   # 任務管理 API
│   │   ├── medical_records.py        # 醫療紀錄 API
│   │   ├── notifications.py          # 通知系統 API
│   │   ├── shelters.py               # 收容所管理 API
│   │   ├── uploads.py                # 檔案上傳 API
│   │   └── users.py                  # 使用者管理 API
│   ├── models/                        # SQLAlchemy ORM 模型
│   │   ├── __init__.py
│   │   ├── animal.py                # 動物模型
│   │   ├── application.py           # 申請模型
│   │   ├── medical_record.py         # 醫療紀錄模型
│   │   ├── pending_registration.py   # 待驗證註冊
│   │   ├── shelter.py               # 收容所模型
│   │   ├── user.py                  # 使用者模型
│   │   └── others.py                # 其他模型
│   ├── services/                      # 業務邏輯層 ★ 核心
│   │   ├── admin_service.py
│   │   ├── animal_service.py
│   │   ├── application_service.py
│   │   ├── attachment_service.py
│   │   ├── audit_service.py
│   │   ├── auth_service.py
│   │   ├── email_service.py
│   │   ├── job_service.py
│   │   ├── medical_record_service.py
│   │   ├── notification_service.py
│   │   ├── permission_service.py
│   │   ├── shelter_service.py
│   │   └── user_service.py
│   ├── tasks/                        # Celery 背景任務
│   ├── templates/                    # 郵件模板
│   └── utils/                         # 工具函數
├── migrations/                       # Alembic 資料庫遷移
├── tests/                             # 測試文件
│   ├── unit/                         # 單元測試 (353 tests)
│   └── integration/                 # 整合測試 (88 tests)
├── htmlcov/                          # 覆蓋率報告 (HTML)
├── config.py                          # 應用配置
├── requirements.txt                   # Python 依賴
├── run.py                             # 應用入口
├── create_test_accounts.py           # 測試帳號腳本
└── .env                              # 環境變數
```


(1) Auth 模

- 功能：使用者註冊、登入、JWT access/refresh token 發放與刷新、登出、電子郵件驗證 (email verification)、密碼重置流程支援。
- 路由範例：
 - POST /auth/register
 - POST /auth/login
 - POST /auth/refresh
 - POST /auth/logout
 - GET /auth/verify?token=
- 說明：
 - 負責身份驗證 (authentication) 與基礎授權 (authorization) 工具。建議使用短生命 access token (JWT) + refresh token (httpOnly cookie 或安全儲存) 並實作 token rotation / revoke 機制。
 - 實作細節建議：使用安全的雜湊演算法 (bcrypt/argon2) 儲存密碼、加入登入失敗次數鎖定與 rate limiting、防止暴力破解；使用 'flask-smorest' 或類似工具做 schema 驗證與 OpenAPI 綁定。
- 安全建議：refresh token 儘量放 httpOnly, Secure cookie；若採用 SPA 並使用 localStorage，需妥善設計 refresh 與 CSRF 保護；管理員高風險操作考慮 MFA。
- 其他：提供 endpoints 支援 email verification 與 password reset (含 expire token)，並在所有重要狀態變更寫入 AuditLog。

(2) Users 模組

- 功能：使用者 CRUD、個人檔案管理、角色管理、primaryShelterId、個資匯出/刪除請求 (job pattern)。
- 路由：
 - GET /users/{id}
 - PATCH /users/{id}
 - POST /data/export
- 說明：實作應包含欄位級別的隱私控制 (PII masking)、匯出/刪除走 job pattern (需審核)，並在敏感操作寫入 AuditLog。

(3) Shelters 模組

- 功能：收容所資料管理、收容所驗證流程、收容所批次匯入 (batch upload) 入口。
- 路由：
 - GET /shelters/{id}
 - POST /shelters/{id}/animals/batch (enqueue job -> 202)
- 說明：batch import 使用 multipart upload + job pattern；需記錄 jobId，worker 由 Celery 處理並回填結果至 Job 表與 Notification。

(4) Animals / Rehomes 模組

- 功能：動物資料與送養刊登管理 (CRUD)、狀態流 (DRAFT, SUBMITTED, PUBLISHED, RETIRED)、圖片/附件關聯管理。
- 路由：
 - GET /animals
 - GET /animals/{id}
 - POST /rehomes
 - PATCH /rehomes/{id}
 - DELETE /rehomes/{id}
- 說明：支援 filters (species, city, q, featured)、pagination/infinite scroll；圖片與 attachments 為 polymorphic metadata，實際檔案存 S3/Minio。

(5) Applications 模組

- 功能：申請 (Application) 建立、查詢、審核流程 (含 assignment)、狀態管理與並發控制 (optimistic locking)。
- 路由：
 - POST /applications
 - GET /applications
 - POST /applications/{id}/review
- 說明：支援 Idempotency-Key header 去重、application.version 作為 optimistic locking 欄位；service 層需驗證申請人資格 (不得為刊登者本人)。

(6) MedicalRecords 模組

- 功能：動物醫療紀錄建立、附件關聯、驗證流程 (verify)。
- 路由：
 - POST /animals/{id}/medical-records
 - POST /medical-records/{id}/verify
- 說明：初始紀錄為 unverified，管理員可標示為 verified；醫療附件應能鏈結至 attachments 並記錄來源與上傳者。

(7) Attachments / Uploads 模組

- 功能：產生 presigned URL、處理 multipart presign (大檔案)、管理 attachment metadata (storageKey, url, ownerType, ownerId)。
- 路由：
 - POST /uploads/presign
 - POST /attachments
 - GET /attachments/{id}
- 說明：presign TTL 建議 5-15 分鐘；上傳成功後前端呼叫 POST /attachments 建立 metadata；後端在 metadata 接收時驗證檔案存在、大小/checksum (若提供) 與擁有權。

(8) Notifications 模組

- 功能：建立與查詢通知紀錄 (DB)、enqueue delivery job、標記已讀。
- 路由：
 - GET /notifications
 - POST /notifications/{id}/mark-read
- 說明：通知產生可由 worker 處理外部投遞 (email/push)；前端可透過 GET /notifications?recipientId= 查詢未讀/已讀狀態。

(9) Jobs 模組

- 功能：統一 Job table、查詢 job 狀態、重試與 metrics、管理與觀察長時間任務生命週期。
- 路由：
- GET /jobs/{jobId}
- 說明：所有長時間任務採 202 + jobId 模式；worker (Celery) 更新 job table (attempts, progress, resultSummary)，並在完成/失敗時發 Notification。

(10) Audit 模組

- 功能：寫入並查詢 AuditLog (actorId, action, before, after, notes, timestamp)。
- 路由：
 - GET /admin/audit
- 說明：在所有重要狀態變更中調用 auditService.log；對於寫入失敗應具備重試/enqueue 機制，且一般使用者不可刪除 audit 紀錄。

(11) Admin 模組

- 功能：管理端專用操作 (資源恢復、審核、報表、系統查詢)。
- 路由：
 - /admin/* (例如 /admin/animals, /admin/users, /admin/applications)
- 說明：需嚴格 RBAC 控制 (角色與 scope)，並記錄所有管理動作至 AuditLog；管理面板功能可搭配限速與多因素驗證。

(12) Health & Observability 模組

- 功能：健康檢查、metrics、tracing、錯誤收集整合。
- 路由：
 - GET /healthz
 - GET /readyz
 - GET /metrics
- 說明：支援 OpenTelemetry traces、Prometheus metrics endpoint 與 Sentry 錯誤監控；在容器化環境提供 readiness/liveness endpoints 以利 k8s probe。

3. 資料庫設計

3.1 資料表

Table : users					
欄位名稱	資料型態	P K	FK	Nullabl e	說明
userId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
email	VARCHAR(320)			X	使用者電子郵件（唯一，作為登入帳號）。
username	VARCHAR(150)			0	使用者公開顯示的帳號或暱稱。
phoneNumber	VARCHAR(32)			0	使用者電話。
firstName	VARCHAR(120)			0	
lastNmae	VARCHAR(120)			0	
region	VARCHAR(100)			0	
address	JSON			0	
role	ENUM('GENERAL_MEMBER' , 'SHELTER_MEMBER' , 'ADMIN')			X	代表使用者的身分，用於授權檢查。 Default: 'GENERAL_MEMBER' 。
verified	TINYINT(1)			X	是否已驗證電子郵件（布林）。 DEFAULT 0。
primaryShelterId	BIGINT UNSIGNED		V	0	使用者所屬或代表的主要收容所 id。(FK)
profilePhotoUrl	VARCHAR(1024)			0	使用者的像檔案的 URL。
settings	JSON			0	儲存使用者偏好設定（例如通知設定）。
passwordHash	VARCHAR(255)			X	密碼雜湊（例如 bcrypt/argon2 的結果），永遠不要儲

					存明文密碼。
passwordChangedAt	DATETIME(6)			0	使用者上次變更密碼的時間。
lastLoginAt	DATETIME(6)			0	用者最近一次成功登入時間（供審計與安全通知）。
failedLoginAttempts	INT			X	Default 0。連續失敗的登入次數，用於實作鎖定策略或 threshold-based 風控。
lockedUntil	DATETIME(6)			0	若因多次失敗而暫時鎖定帳號，記錄解鎖時間（timestamp），NULL 表示未鎖定。
createAt	DATETIME(6)			X	建立時間戳。
updateAt	DATETIME(6)			X	更新時間戳。（預設為建立時間）
deleteAt	DATETIME(6)			0	軟刪除時間戳；若不為 NULL 表示帳號已被刪除但保留紀錄。

Table : shelters					
欄位名稱	資料型態	P K	FK	Nullabl e	說明
shelterId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
name	VARCHAR(255)			X	使用者電子郵件（唯一，作為登入帳號）。
slug	VARCHAR(255)			0	使用者暱稱或帳號名稱，可為空。
contactEmail	VARCHAR(320)			X	
contactPhone	VARCHAR(32)			X	
address	JSON			X	JSON 格式的地址欄位，可包含街道、城市、縣市與郵遞

					區號，便於顯示與地圖整合。
region	VARCHAR(100)			0	
verified	TINYINT(1)			X	收容所是否通過系統管理員審核（布林），影響是否可被標註為「官方認可」。
primaryAccountUserId	BIGINT UNSIGNED		V	0	指向管理此收容所的主要使用者(userId) (FK)，用於角色與通知路徑。
createdAt	DATETIME(6)			X	建立時間戳。
updatedAt	DATETIME(6)			X	更新時間戳。（預設為建立時間）
deletedAt	DATETIME(6)			0	軟刪除時間戳；若不為 NULL 表示帳號已被刪除但保留紀錄。

Table : pending_registrations					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
pending_id	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
email	VARCHAR(255)			X	使用者電子郵件（唯一，作為登入帳號）。
username	VARCHAR(255)			0	使用者暱稱或帳號名稱，可為空。
phone_number					
region	VARCHAR(100)			0	
address	JSON			0	
password_hash					
verification_co					

de_hash					
code_expires_at					
attempts					
resend_count					
client_ip					
user_agent					
createdAt	DATETIME(6)			X	建立時間戳。
updatedAt	DATETIME(6)			X	更新時間戳。(預設為建立時間)

Table : animals					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
animalId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
name	VARCHAR(200)			0	動物的稱呼或暱稱，供前端卡片與列表顯示。
species	ENUM(' CAT', ' DOG')			0	動物種類 (ENUM)，便於分類與篩選。
breed	VARCHAR(200)			0	品種欄位，用於細分搜尋與顯示。
color	VARCHAR(100)			0	
sex	ENUM(' MALE', ' FEMALE', ' UNKNOWN')			0	性別 (ENUM)。
dob	DATE			0	出生日期 (若已知)，可用於計算年齡。
description	TEXT			0	文字敘述，包括性情、背景故事或特殊需求。

status	ENUM(' DRAFT' , ' SUBMITTED' , ' PUBLISHED' , ' RETIRED')			X	工作流程狀態 (DRAFT: 草稿, SUBMITTED: 已送審, PUBLISHED: 上架, RETIRED: 下架) , 控制是否曝光與可申請。
shelterId	BIGINT UNSIGNED		V	0	若動物由收容所管理, 記錄該 shelterId (FK) 。
ownerId	BIGINT UNSIGNED		V	0	若有原持有人或個人刊登者, 記錄 userId 。
medicalSummary	TEXT			0	針對醫療資訊的摘要 (例如疫苗、手術、慢性病) , 常用於列表快覽。
rejection_reason	TEXT			0	拒絕審核相關欄位
rejected_at	DATETIME(6)			0	拒絕審核相關欄位
rejected_by	BIGINT UNSIGNED		V	0	拒絕審核相關欄位
createdBy	BIGINT UNSIGNED		V	X	建立此動物紀錄的使用者 userId (FK) , 便於追溯與權限判斷。
createdAt	DATETIME(6)			X	建立時間戳。
updatedAt	DATETIME(6)			X	更新時間戳。(預設為建立時間)
deletedAt	DATETIME(6)			0	軟刪除時間戳; 若不為 NULL 表示帳號已被刪除但保留紀錄。

Table : animalImages					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
animalImageId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
animalId	VARCHAR(320)			X	外鍵, 指向所屬 animal, 用

					於查詢該動物所有圖片。
storageKey	VARCHAR(150)			X	object storage (MinIO) 內的物件鍵，用於直接上傳/下載。
url	VARCHAR(2048)			X	圖片對外存取的 URL (可為 CDN URL 或 presigned URL)，供前端顯示。
mimeType	VARCHAR(128)			0	檔案的類型。
width	INT			0	影像寬高，可用於產生佔位圖或 lazy loading 決策。
height	INT			0	影像寬高，可用於產生佔位圖或 lazy loading 決策。
order	INT			X	圖片的顯示順序 (整數)，用於排序，Default 0。
createdAt	DATETIME(6)			X	上傳/紀錄時間。

Table : applications					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
applicantId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
applicantId	BIGINT UNSIGNED			X	申請人 userId (FK)，代表誰提出申請。
animalId	BIGINT UNSIGNED			X	申請對象的 animalId (FK)。
type	ENUM(' ADOPTION', ' REHOME')			X	申請類型 (ADOPTION 或 REHOME)。
status	ENUM(' PENDING', ' UNDER_REVIEW', ' APPROVED', ' REJECTED', ' WITHDRAWN')			X	申請狀態 (PENDING / UNDER_REVIEW / APPROVED / REJECTED / WITHDRAWN)。

submittedAt	DATETIME(6)			0	提交與審核時間，用於審計與 SLA 計算。
reviewedAt	DATETIME(6)			0	提交與審核時間，用於審計與 SLA 計算。
reviewNotes	TEXT			0	核者留下的文字備註（理由或拒絕原因）。
assigneeId	BIGINT UNSIGNED			0	負責處理該申請的審核者 use rId (FK)。
version	INT			X	樂觀鎖欄位 (int)，在更新時檢查以避免競爭條件。
idempotencyKey	VARCHAR(255)			0	用於 POST 去重，避免重複建立相同申請（例如 client 重試情況）。
attachments	JSON			0	JSON 陣列，儲存申請所附檔案的 metadata 或 attachment ids。
contact_phone	VARCHAR(32)			0	申請人詳細資料欄位，聯絡資訊。
contact_address	VARCHAR(500)			0	申請人詳細資料欄位，聯絡資訊。
occupation	VARCHAR(100)			0	職業。
housing_type	VARCHAR(50)			0	住宅類型（公寓、透天厝等）。
has_experience	TINYINT(1)			X	是否有飼養經驗，defulat = 0
reason	TEXT			0	領養原因。
notes	TEXT			0	其他備註。
createdAt	DATETIME(6)			X	建立時間戳。
updated_at	DATETIME(6)			X	更新時間戳。（預設為建立時間）
deleted_at	DATETIME(6)			0	軟刪除時間戳；若不為 NULL 表示帳號已被刪除但保留紀

				錄。
--	--	--	--	----

Table : medicalRecords					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
medicalRecordId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
animalId	BIGINT UNSIGNED		V	X	對應該紀錄所屬的animalId。
recordType	ENUM(' TREATMENT', ' CHECKUP', ' VACCINE', ' SURGERY', ' OTHER')			0	醫療紀錄類型，用於分類與過濾。
date	DATE			0	紀錄發生日期（若已知）。
provider	VARCHAR(255)			0	提供醫療服務者或醫療機構名稱。
details	TEXT			0	記錄詳述（文本），包含診斷、處置、醫囑等。
attachments	JSON			0	JSON 陣列，連結相關檔案（例如病歷表、影像、檢驗結果）。
verified	TINYINT(1)			X	布林，表示該紀錄是否已經被授權人員(Admin)驗證。
verifiedBy	BIGINT UNSIGNED		V	0	驗證者的 userId (FK)，在驗證時填入，用於追蹤誰完成驗證。
createdBy	BIGINT UNSIGNED			0	建立此紀錄的 userId (F K)，通常是上傳或輸入紀錄的人。
createdAt	DATETIME(6)			X	建立時間戳。
updatedAt	DATETIME(6)			X	更新時間戳。（預設為建立時間）

deletedAt	DATETIME(6)			0	軟刪除時間戳；若不為 NULL 表示帳號已被刪除但保留紀錄。
-----------	-------------	--	--	---	--------------------------------

Table : notifications					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
notificationId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
recipientId	BIGINT UNSIGNED		V	X	接收者 userId (FK)，是查詢通知的主要索引。
actorId	BIGINT UNSIGNED			0	執行該動作的 userId (可為 NULL，例如系統通知)。
type	VARCHAR(128)			X	通知類型字串 (例如 'application_submitted')，用於選擇 template 與處理邏輯。
payload	VARCHAR(128)			0	JSON，儲存模板變數或原始事件資料，供通知服務解析並送出。
read	TINYINT(1)			X	是否已讀 (布林)；用於前端顯示未讀 badge。
createdAt	DATETIME(6)			X	建立時間。
readdAt	DATETIME(6)			0	讀取的時間。

Table : jobs					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
jobId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵。
type	VARCHAR(320)			X	工作類型 (例如 'shelter_ba

					tch_import')，用於 worker 路由與 metrics。
status	ENUM(' PENDING', ' RUNNING', ' SUCCEEDED', ' FAILED')			X	動作類別字串（例如 ' application.approve', ' user.update' ），便於篩選與搜尋。
payload	JSON			0	被操作的資源類型（例如 ' application', ' animal' ）。
resultSummary	JSON			0	執行結果摘要（例如 { total:100, ok:95, failed:5 } ）。
createdBy	BIGINT UNSIGNED		V	0	建立這個 job 的 userId（或 NULL 表示系統）。
createdAt	DATETIME(6)			X	建立的時間。
startedAt	DATETIME(6)			0	worker 開始處理時間。
finishedAt	DATETIME(6)			0	結束時間。用於 SLA/時長統計。
attempts	INT				重試次數，用於防止無限重試或判定重試策略。

Table : attachments					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
attachmentId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵
ownerType	VARCHAR(64)			x	表示此附件屬於哪種類型的實體（例如 ' animal', ' application', ' medical_record' ），用於在應用層解析擁有者。
ownerId	BIGINT UNSIGNED			X	與 ownerType 配合，表示具體的資源 id（例如 animal.id 或 application.id）。此欄不是 DB FK，而是應用層維

					護關聯。
storageKey	VARCHAR(1024)			X	在 object storage 中的鍵值，用於直接上傳/下載。
url	VARCHAR(2048)			X	可供前端顯示或下載的完整 URL（可能為經由 CDN 的網址）。
filename	VARCHAR(1024)			0	原始檔名，方便展示與下載時使用者識別。
mimeType	VARCHAR(128)			0	檔案類型（image/jpeg, application/pdf 等），可用於安全檢查或顯示判斷。
size	INT			0	檔案大小（位元組），可用於 quota 與顯示。
meta_data	JSON			0	額外元資料。
createdBy	BIGINT UNSIGNED		V	0	userId（FK），方便追蹤上傳者。
createdAt	DATETIME(6)			X	上傳時間。
deletedAt	DATETIME(6)			0	軟刪除時間。

Table：auditLogs					
欄位名稱	資料型態	P K	F K	Nullabl e	說明
auditLogId	BIGINT UNSIGNED AUTO_INCREMENT	V		X	主鍵
actorId	VARCHAR(320)		V	0	執行者 userId（可為 NULL，表示系統動作）。
action	VARCHAR(150)			X	動作類別字串（例如 'application.approve'，'user.update'），便於篩選與搜尋。
targetType	VARCHAR(128)			0	被操作的資源類型（例如 'ap

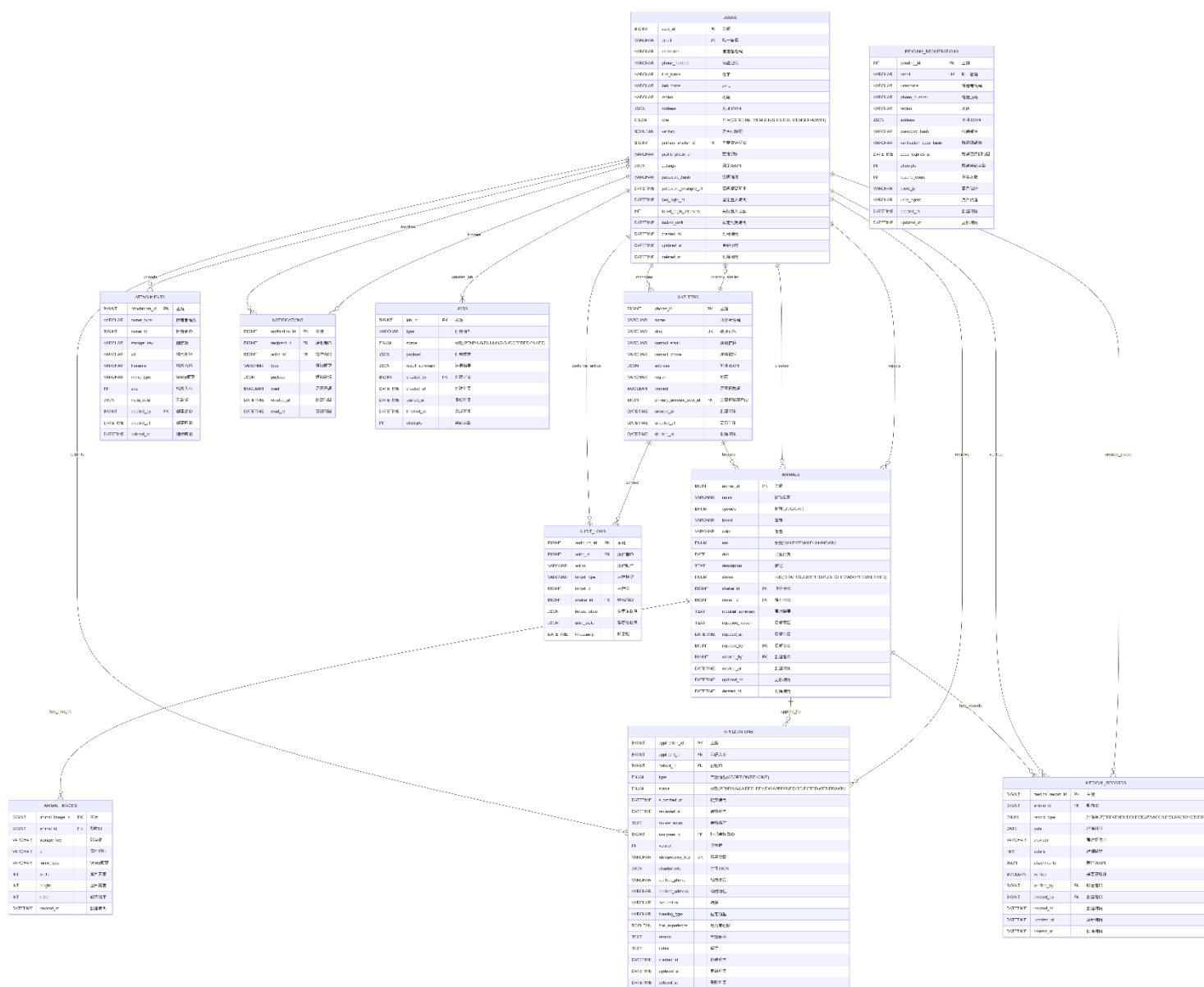
					plication'、'animal') 。
targetId	BIGINT UNSIGNED			0	被操作的資源 id。
shelterId	BIGINT UNSIGNED			0	若該動作與某 shelter 相關，記錄該 shelterId 以便過濾。
beforeState	JSON			0	JSON 快照，紀錄變更前後的欄位值，用於鑑識與合規查詢。
afterState	JSON			0	JSON 快照，紀錄變更前後的欄位值，用於鑑識與合規查詢。
timestamp	DATETIME(6)			X	事件發生時間，預設為 CURRENT_TIMESTAMP(6)。

註：資料型態以mySQL去撰寫。

3.3 資料表間的關聯關係

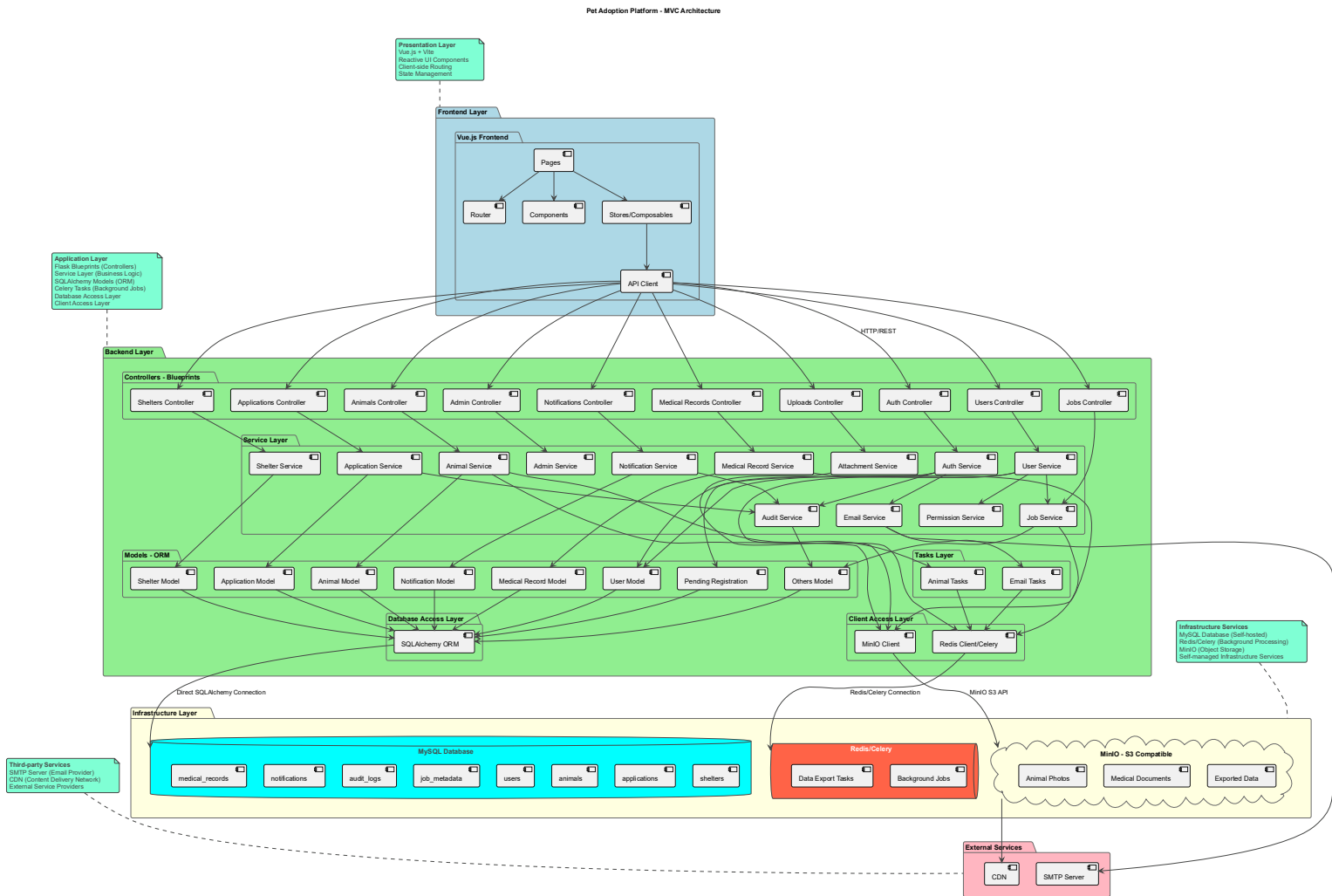
- User (管理者) 對 Shelter 為一對多 (User 可管理多個 Shelter)
- Shelter 對 User 為一對多 (多位 User 可指向同一 Shelter, 作為主要所屬)
- Shelter 對 Animal 為一對多 (收容所可能有多隻動物)
- User (owner) 對 Animal 為一對多 (ownerId)
- User (created_by) 對 Animal 為一對多 (建立者)
- User (rejected_by) 對 Animal 為一對多 (拒絕審核者)
- Animal 對 AnimalImages 為一對多, 圖片為子資源
- User (applicant) 對 Applications 為一對多 (申請者)
- Animal 對 Applications 為一對多 (被申請動物)
- User (assignee) 對 Applications 為一對多 (審核者)
- Animal 對 MedicalRecords 為一對多 (醫療記錄)
- User (verified_by) 對 MedicalRecords 為一對多 (驗證者)
- User (created_by) 對 MedicalRecords 為一對多 (建立者)
- User (recipient) 對 Notifications 為一對多 (接收者)
- User (actor) 對 Notifications 為一對多 (執行者)
- User (created_by) 對 Jobs 為一對多 (任務建立者)
- User (actor) 對 AuditLogs 為一對多 (操作者)
- Shelter 對 AuditLogs 為一對多 (相關收容所)
- User (created_by) 對 Attachments 為一對多 (上傳者)
- MedicalRecord/Application 對 Attachments 為一對多 (polymorphic)

3.4 ERD



註：若圖不清楚可參考 `erd.png`。

4. 系統架構圖



註：若圖不清楚可參考 [mvc.svg](#)。